Leçon 6 : Widgets de Base

Pour Commencer **6-2**
Styliser Votre Application **6-5**
Définir une Classe de Thème **6-6**
Application du Thème 6-7
Changer de Thème **6-9**
Implémentation des Changements d'État du Thème 6-10
Créer des Boutons Personnalisés pour Changer la Couleur et le Mode **6-11**
Création du Bouton de Thème 6-11
Création du Bouton de Couleur 6-13
Ajouter des Boutons d'Action à la Barre d'Application 6-14
Comprendre la Structure de l'App et la Navigation **6-15**
Utiliser Scaffold 6-16
Naviguer Entre les Pages **6-21**
Créer des Cartes Personnalisées **6-22**
Composer la Carte de Catégorie **6-23**
Composer la Carte de Publication (Post Card) **6-29**
Ajout des Widgets Enfants 6-32
Composer la Carte de Restaurant en Paysage 6-34
Composition des Widgets Enfants du Restaurant 6-36
Points Clés **6-39**

Plongez dans l'univers de Flutter, où tout est un widget ! Cette leçon dévoile trois catégories fondamentales de widgets essentielles pour :

- **La structure et la navigation**
- **L'affichage d'informations**
- **Le positionnement des widgets**

À la fin de cette leçon, vous construirez une application de réseautage culinaire appelée **Yummy**. Vous utiliserez divers widgets pour créer trois onglets distincts : **Catégorie**, **Publication** et **Restaurant**.

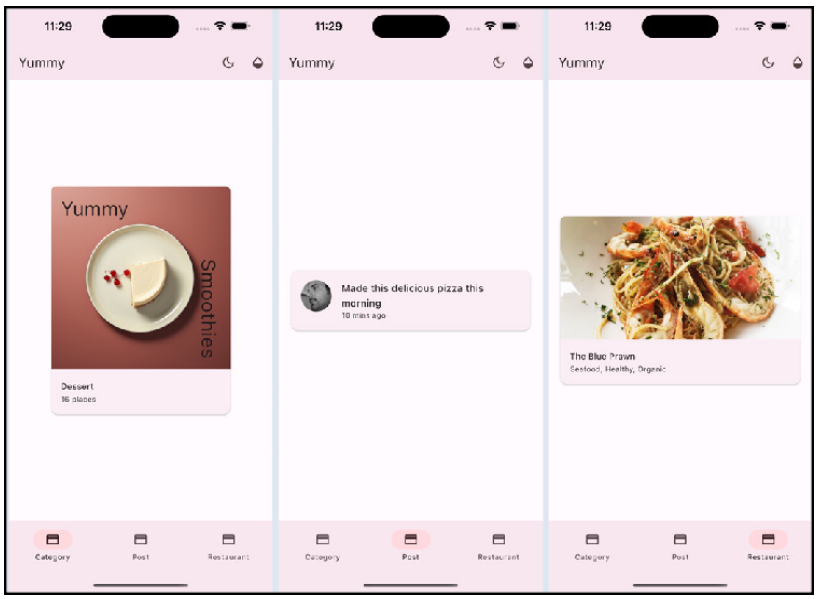

Figure 1 : Aperçu de l'application Yummy

Prêt ? Commencez par jeter un œil au projet de départ.

Pour Commencer

Commencez par télécharger le projet de cette leçon depuis le dépôt des ressources sur **Google Drive** `https://bit.ly/47xnDIV`.

Localisez le dossier **projects** et ouvrez **starter**. Naviguez jusqu'à **pubspec.yaml** et appuyez sur **Pub get** pour obtenir toutes vos dépendances Flutter.

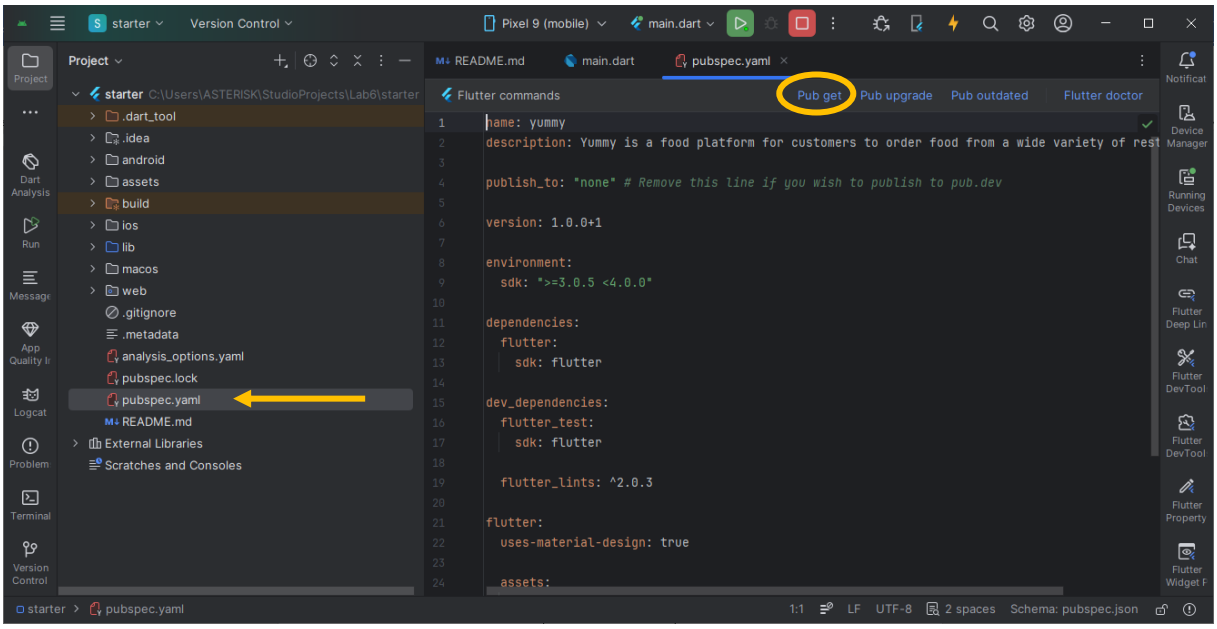

Figure 2 : Téléchargement des dépendances

Lancez l'application, et vous verrez une barre d'application et un texte simple :

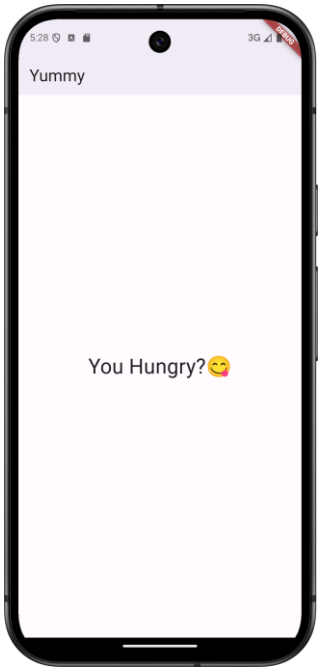

Figure 3 : Écran de démarrage de l'application

lib/main.dart sert de rampe de lancement pour toute application Flutter. Ouvrez-le pour voir :

```
import 'package:flutter/material.dart';
void main() {
  // 1
  runApp(const Yummy());
}
class Yummy extends StatelessWidget {
  // TODO: Setup default theme
  // 2
  const Yummy({super.key});
  // TODO: Add changeTheme above here
  @override
  Widget build(BuildContext context) {
    const appTitle = 'Yummy';
    // TODO: Setup default theme
    //3
    return MaterialApp(
      title: appTitle,
      //debugShowCheckedModeBanner: false,
      // Uncomment to remove Debug banner
      // TODO: Add theme
      // TODO: Replace Scaffold with Home widget
      // 4
      home: Scaffold(
        appBar: AppBar(
          // TODO: Add action buttons
          elevation: 4.0,
          title: const Text(
            appTitle,
            style: TextStyle(fontSize: 24.0),
          ),
        ),
        body: const Center(
          child: Text(
            'You Hungry?  ',
            style: TextStyle(fontSize: 30.0),
          ),
        ),
      ),
    );
  }
}
```

Prenez un moment pour explorer ce que fait le code :

1. **Initialisation du Widget** : Chaque voyage avec Flutter commence par un widget. La fonction runApp() initialise l'application en acceptant le widget racine, en l'occurrence une instance de Yummy.
2. Chaque widget doit remplacer la méthode `build()`.
3. Le widget Yummy commence par composer un widget MaterialApp pour lui donner l'apparence du système **Material Design**. Voir https://material.io pour plus de détails.
4. **Scaffold** définit la structure visuelle de l'application, contenant une AppBar et un corps pour commencer.

Styliser Votre Application

Flutter, étant multiplateforme, prend en charge les systèmes de design Material Design d'Android et Cupertino d'iOS.

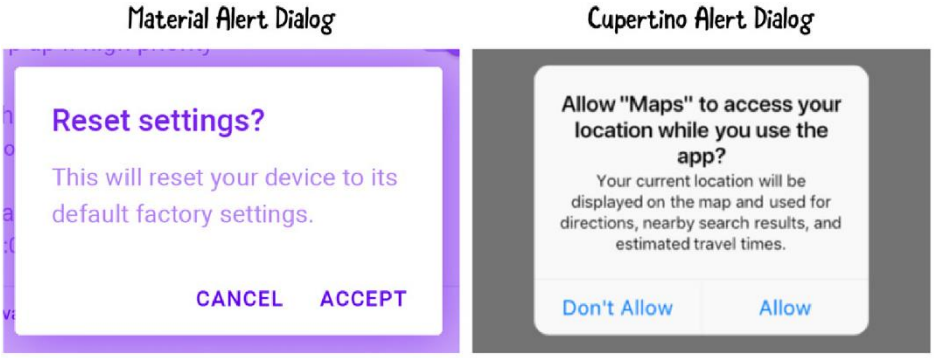

Figure 4 : Systèmes de design Material et Cupertino

Android utilise le système **Material Design**, que vous importeriez comme ceci :

```
import 'package:flutter/material.dart';
```

iOS utilise le système **Cupertino**. Voici comment vous l'importeriez :

```
import 'package:flutter/cupertino.dart';
```

Tout au long de ces leçons, vous apprendrez à utiliser le système Material Design. Vous constaterez que son apparence et sa convivialité sont hautement personnalisables !

Remarque : Basculer entre Material et Cupertino dépasse le cadre de ces leçons. Pour plus d'informations sur ce que ces packages offrent en termes de composants d'interface utilisateur, consultez :

- Composants d'interface Material : https://flutter.dev/docs/development/ui/widgets/material

- Composants d'interface Cupertino : https://flutter.dev/docs/development/ui/widgets/cupertino

Maintenant que vous avez choisi un design, vous allez définir un thème pour votre application dans la section suivante.

Définir une Classe de Thème

Donnez du piquant à votre application avec un thème personnalisé ! Avec **Material 3**, la gestion des thèmes est simplifiée, en se concentrant sur la définition des variations de couleurs.

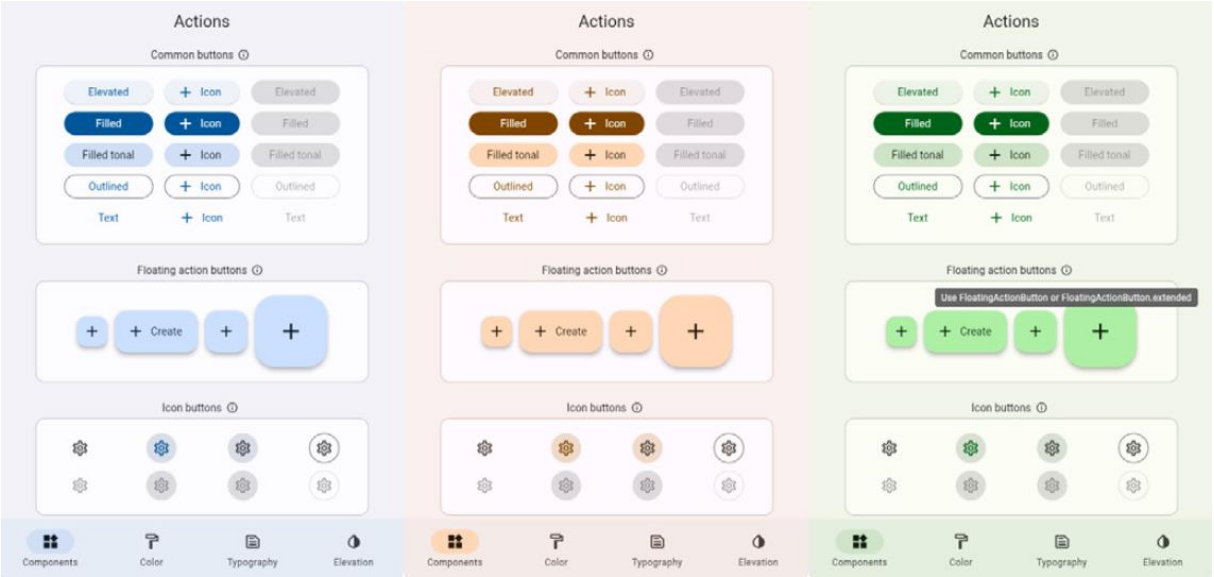

Figure 5 : Exemple de sélecteur de thème de couleur

Ouvrez **lib/constants.dart** et examinez le code inclus dans votre projet de départ :

```
import 'package:flutter/material.dart';
enum ColorSelection {
  // 1
  deepPurple('Deep Purple', Colors.deepPurple),
  purple('Purple', Colors.purple),
  indigo('Indigo', Colors.indigo),
  blue('Blue', Colors.blue),
  teal('Teal', Colors.teal),
  green('Green', Colors.green),
  yellow('Yellow', Colors.yellow),
  orange('Orange', Colors.orange),
  deepOrange('Deep Orange', Colors.deepOrange),
  pink('Pink', Colors.pink);
```

```
  // 2
  const
    ColorSelection(
    this.label,
    this.color,
  );
  final String label;
  final Color color;
}
```

L'énumération `ColorSelection` permet aux utilisateurs de sélectionner et de personnaliser l'apparence de l'application avec :

1. Des options de couleurs structurées. Le nom listé (par exemple "Deep Purple") est ce qui sera affiché.
2. Chaque option a un **`libellé`** et un objet de **`couleur`**.

Maintenant, vous allez apprendre à appliquer les thèmes de couleur à votre application.

Application du Thème

Dans **main.dart**, importez vos thèmes de couleur prédéfinis :

```
import 'constants.dart';
```

Localisez **`// TODO: Setup default theme`** et remplacez-le par le code suivant pour établir votre mode de thème par défaut et votre couleur primaire, ignorez les lignes ondulées rouges :

```
ThemeMode themeMode = ThemeMode.light; // Manual theme toggle
ColorSelection colorSelected = ColorSelection.pink;
```

Ensuite, localisez le commentaire **`// TODO : Add theme`** et insérez le code suivant pour appliquer vos configurations de thème :

```
themeMode: themeMode,
theme: ThemeData(
  colorSchemeSeed: colorSelected.color,
  useMaterial3: true,
  brightness: Brightness.light,
),
```

```
darkTheme: ThemeData(
  colorSchemeSeed: colorSelected.color,
  useMaterial3: true,
  brightness: Brightness.dark,
),
```

Cet extrait de code définit le mode de thème global. Il définit à la fois les thèmes **clair** et **sombre** en utilisant la couleur que vous avez spécifiée précédemment, assurant une apparence visuelle cohérente et adaptable dans toute votre application.

Comme le thème peut changer, vous devez supprimer **`const`** aux deux endroits suivants :

```
runApp(const Yummy());

...

const Yummy({super.key});
```

Puisque vous utilisez un **`StatelessWidget`**, vous devez vous assurer que tous les champs sont de type **`final`** si vous voulez utiliser un constructeur **`const`**.

```
final ThemeMode themeMode = ThemeMode.light; // Manual theme toggle
final ColorSelection colorSelected = ColorSelection.pink;
```

Enregistrez vos modifications et effectuez un redémarrage à chaud (**hot restart**).

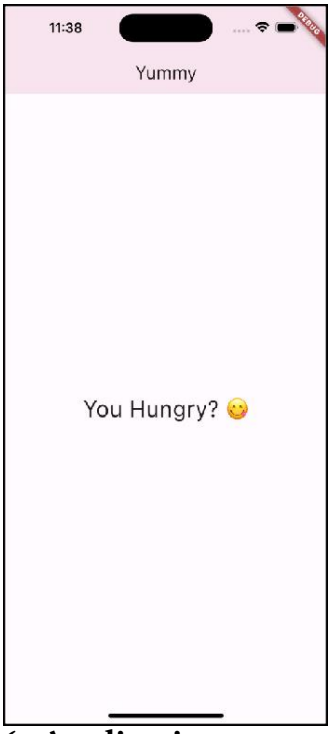

Figure 6 : Application en mode clair

Localisez `//Manual theme toggle` et remplacez `light` par `dark` pour observer les variations de thème. Assurez-vous de faire un redémarrage à chaud (**hot restart**).

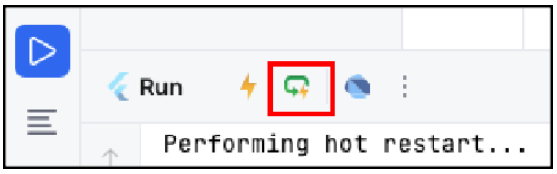

Figure 7 : Commutation de thème

Les deux thèmes ressemblent à ceci :

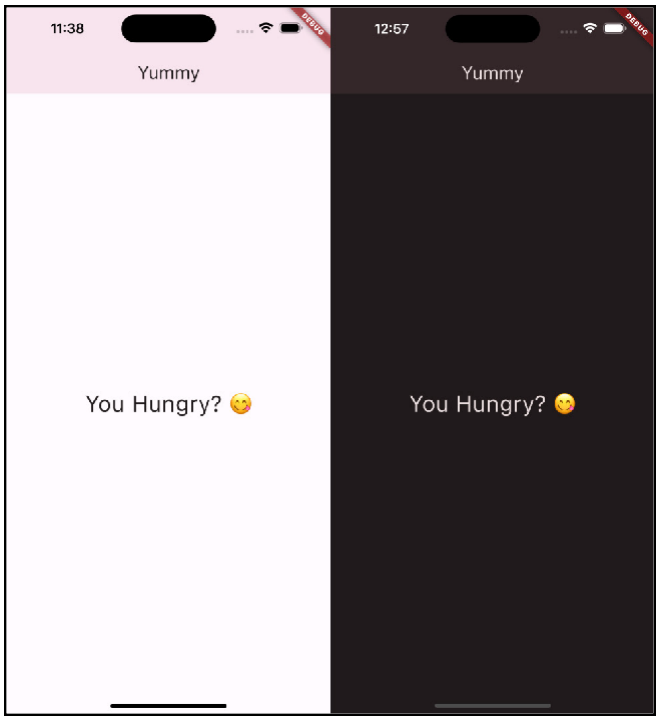

Figure 8 : Comparaison des thèmes clair et sombre

Ensuite, vous allez créer un moyen permettant aux utilisateurs de basculer entre les modes clair et sombre et de sélectionner un thème de couleur personnalisé.

Changer de Thème

Pour permettre le changement de thème dans votre application, vous devez gérer l'état en convertissant le widget `Yummy` en `StatefulWidget`. La bonne nouvelle est qu'au lieu de le faire manuellement, vous pouvez utiliser un raccourci de menu contextuel pour le faire automatiquement.

Faites un clic droit sur le nom de la classe `Yummy`. Cliquez ensuite sur **Show Context Actions** dans le menu qui apparaît :

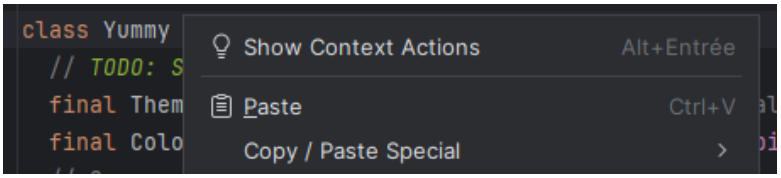

Figure 9 : Menu contextuel pour convertir en StatefulWidget

Sélectionnez **Convert to StatefulWidget**.

Figure 10 : Résultat de la conversion

Il y a maintenant deux classes :

```
class Yummy extends StatefulWidget {
  ...
  @override
  State<Yummy> createState() => _YummyState_();
}
class _YummyState extends State<Yummy> {
  ...
  @override
  Widget build(BuildContext context) {
    ...
  }
}
```

Quelques éléments à noter dans le code ci-dessus :

- La refactorisation a converti **`Yummy`** de **`StatelessWidget`** en **`StatefulWidget`**. Elle a ajouté une implémentation de **`createState()`**.
- La refactorisation a également créé la classe d'état **`_YummyState`**. Elle stocke des données mutables qui peuvent changer au cours de la durée de vie du widget.

N'est-ce pas pratique quand il y a un moyen automatique de gagner du temps ? Ensuite, vous allez implémenter les changements d'état du thème.

Implémentation des Changements d'État du Thème

Dans la classe **`_YummyState`**, localisez **`// TODO: Add changeTheme above here`** et remplacez-le par les fonctions suivantes :

```
void changeThemeMode(bool useLightMode) {
  setState(() {
    // 1
    themeMode = useLightMode
      ? ThemeMode.light //
      : ThemeMode.dark;
  });
}
void changeColor(int value) {
  setState(() {
    // 2
    colorSelected = ColorSelection.values[value];
  });
}
```

Voici comment le code fonctionne :

1. Met à jour le mode de thème en fonction de la sélection de l'utilisateur.
2. Met à jour la couleur du thème en fonction de la sélection de l'utilisateur.

L'appel de ces fonctions mettra à jour le thème ou la couleur de votre application. Maintenant, vous devez créer des boutons personnalisés pour mettre à jour le thème.

Créer des Boutons Personnalisés pour Changer la Couleur et le Mode

Il est maintenant temps de créer deux boutons qui permettront à vos utilisateurs de :

- Basculer entre le mode clair et sombre.
- Sélectionner le thème de couleur de l'ensemble de l'application.

Création du Bouton de Thème

Vous allez créer un bouton pour basculer entre le mode clair et sombre. Dans le répertoire **lib**, créez un nouveau dossier appelé **components** et créez un nouveau fichier nommé **theme_button.dart** dans ce répertoire. Ajoutez le code suivant :

```
import 'package:flutter/material.dart';
class ThemeButton extends StatelessWidget {
  // 1
  const ThemeButton({
    Key? key,
    required this.changeThemeMode,
  }) : super(key: key);
  // 2
  final Function changeThemeMode;
  @override
  Widget build(BuildContext context) {
    // 3
    final isBright = Theme.of(context).brightness ==
Brightness.light;
    // 4
    return
      IconButton(
      icon: isBright
          ? const Icon(Icons.dark_mode_outlined) //
          : const Icon(Icons.light_mode_outlined),
      // 5
      onPressed: () => changeThemeMode(!isBright),
    );
  }
}
```

Prenez un moment pour parcourir le code :

1. Le widget **`ThemeButton`** est initialisé avec un constructeur nécessitant un paramètre de fonction **`changeThemeMode`**.
2. **`changeThemeMode`** est une fonction de rappel passée comme paramètre pour être appelée lorsque l'utilisateur appuie sur le bouton. Cette fonction notifie le widget parent du changement de luminosité, lui permettant d'ajuster le thème en conséquence.
3. **`isBright`** est un booléen qui vérifie si la luminosité du thème actuel est claire.
4. Un widget **`IconButton`** qui affichera l'icône de mode clair ou sombre en fonction du booléen **`isBright`**.
5. **`IconButton`**, lorsqu'il est pressé, bascule la luminosité du thème en invoquant **`changeThemeMode`**.

Ensuite, vous allez créer un bouton pour que les utilisateurs puissent sélectionner leur couleur préférée afin d'appliquer le thème de toute l'application.

Création du Bouton de Couleur

Dans le répertoire **lib/components**, créez un nouveau fichier appelé **color_button.dart** et ajoutez le code suivant :

```
import 'package:flutter/material.dart';
import '../constants.dart';
class ColorButton extends StatelessWidget {
  // 1
  const ColorButton({
    super.key,
    required this.changeColor,
    required this.colorSelected,
  });
  // 2
  final void Function(int) changeColor;
  final ColorSelection colorSelected;
  @override
  Widget build(BuildContext context) {
    // 3
    return PopupMenuButton(
      icon: Icon(
        Icons.opacity_outlined,
        color:
        Theme.of(context).colorScheme.onSurfaceVariant,
      ),
      // 4
      shape: RoundedRectangleBorder(
        borderRadius:
        BorderRadius.circular(10),
      ),
      // 5
      itemBuilder: (context) {
        // 6
        return List.generate(
          ColorSelection.values.leng
          th, (index) {
            final currentColor =
            ColorSelection.values[index];
            // 7
            return PopupMenuItem(
              value: index,
              enabled: currentColor != colorSelected,
```

```
              child: Wrap(
                children:
                  [
                  Padding(
                    padding: const EdgeInsets.only(left:
                    10), child: Icon(
                      Icons.opacity_outlined,
                      color:
                      currentColor.color,
                    ),
                  ),
                  Padding(
                    padding: const EdgeInsets.only(left:
                    20), child: Text(currentColor.label),
                  ),],),);},);},
      // 8
      onSelected: changeColor,
    );
  }
}
```

Prenez un moment pour parcourir le code :

1. Initialise **`ColorButton`** avec le rappel et la couleur requis.
2. La propriété **`changeColor`** est un rappel pour gérer la sélection de couleur, et **`colorSelected`** est la couleur actuellement sélectionnée.
3. Crée un bouton qui affiche un menu.
4. Applique des coins arrondis au menu contextuel.
5. Génère les éléments du menu.
6. Crée une liste d'options de couleur à partir de **`ColorSelection`**.
7. Configure chaque élément de menu avec une icône et un texte.
8. Appelle **`changeColor`** lorsqu'un élément est sélectionné.

Maintenant que vous avez créé les boutons, il est temps de les ajouter à votre application.

Ajouter des Boutons d'Action à la Barre d'Application

Dans **main.dart**, ajoutez les imports suivants :

```
import 'components/theme_button.dart';
import 'components/color_button.dart';
```

Ensuite, localisez `// TODO: Add action buttons` et remplacez-le par le code suivant :

```
actions: [
  ThemeButton(
    changeThemeMode: changeThemeMode,
  ),
  ColorButton(
    changeColor: changeColor,
    colorSelected: colorSelected,
  ),
],
```

Avec un redémarrage à chaud, vous devriez voir les deux nouveaux boutons en haut à droite. Essayez de basculer entre le mode clair et sombre et de changer le thème de couleur.

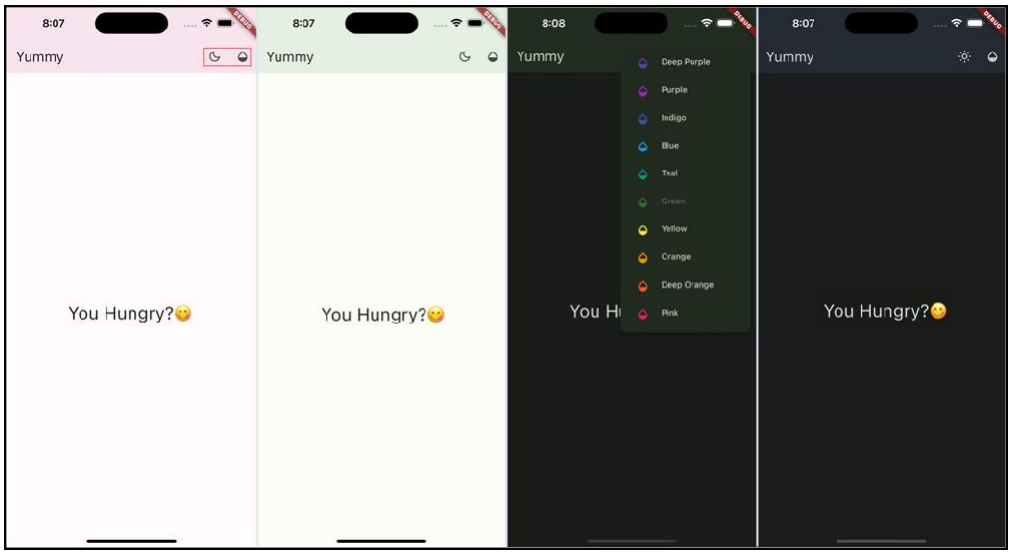

Figure 11 : Application avec les boutons de thème et de couleur

Ensuite, vous allez apprendre un aspect important de la construction d'une application --- comprendre quelle structure d'application utiliser.

Comprendre la Structure de l'App et la Navigation

Établir la structure de votre application dès le départ est important pour l'expérience utilisateur. Appliquer la bonne structure de navigation permet à vos utilisateurs de naviguer facilement dans les informations de votre application.

Yummy utilise le widget Scaffold pour sa structure d'application de départ. Scaffold est l'un des widgets Material les plus couramment utilisés dans Flutter. Ensuite, vous apprendrez à l'implémenter dans votre application.

Utiliser Scaffold

Le widget Scaffold implémente tous vos besoins de base en matière de structure de mise en page visuelle. Il est composé des parties suivantes :

- AppBar
- BottomSheet
- BottomNavigationBar
- Drawer
- FloatingActionButton
- SnackBar

Scaffold a beaucoup de fonctionnalités prêtes à l'emploi !

Le diagramme suivant représente certains des éléments mentionnés précédemment ainsi que les options de navigation gauche et droite :

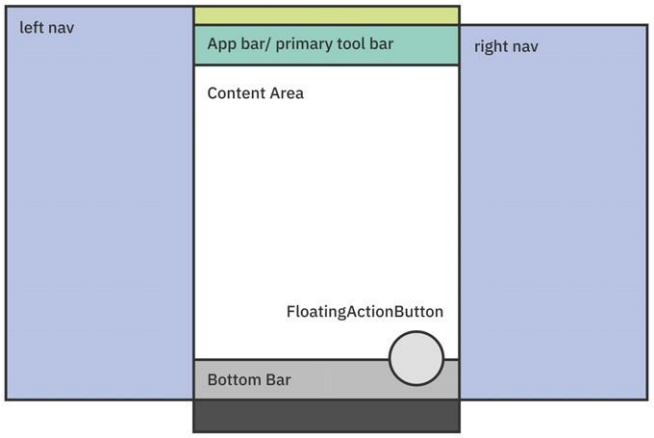

Figure 12 : Diagramme des composants de Scaffold

Pour plus d'informations, consultez la documentation de Flutter sur les **widgets des composants Material**, y compris la structure de l'application et la navigation : https://flutter.dev/docs/development/ui/widgets/material

Maintenant, il est temps d'ajouter plus de fonctionnalités.

Configuration du Widget Home

Au fur et à mesure que vous construisez des applications à grande échelle, vous commencerez à composer un escalier de widgets. Les widgets composés d'autres widgets peuvent devenir très longs et désordonnés. C'est une bonne idée de diviser vos widgets en fichiers séparés pour une meilleure lisibilité.

Pour éviter que votre code ne devienne trop compliqué, vous allez créer le premier de ces fichiers séparés maintenant.

Votre prochaine étape consiste à déplacer le code de **main.dart** vers un nouveau **`StatefulWidget`** nommé **`Home`**. Dans le répertoire **lib**, créez un nouveau fichier appelé **home.dart** et ajoutez ce qui suit :

```
import 'package:flutter/material.dart';
import 'components/theme_button.dart';
import 'components/color_button.dart';
import 'constants.dart';

class Home extends StatefulWidget {
  const Home({
    super.key,
    required this.changeTheme,
    required this.changeColor,
    required this.colorSelected,
  });
  final void Function(bool useLightMode) changeTheme;
  final void Function(int value) changeColor;
  final ColorSelection colorSelected;
  @override
  State<Home> createState() => _HomeState();
}
class _HomeState extends State<Home> {
  // TODO: Track current tab
  // TODO: Define tab bar destinations
  @override
  Widget build(BuildContext context) {
    // TODO: Define pages
    return Scaffold(
      appBar: AppBar(
        elevation: 4.0,
        backgroundColor:
Theme.of(context).colorScheme.background
        , actions: [
          ThemeButton(
            changeThemeMode: widget.changeTheme,
          ),
          ColorButton(
            changeColor: widget.changeColor,
            colorSelected:
            widget.colorSelected,
          ),
        ],
      ),
```

```
      // TODO: Switch between pages
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(
            'You Hungry?  ',
            style: Theme.of(context).textTheme.displayLarge,
          ),
      ),
      // TODO: Add bottom navigation bar
    );
  }
}
```

Vous avez simplement copié le Scaffold de **main.dart** dans un nouveau widget dans **home.dart**.

> **Remarque** : Rappelez-vous si vous voyez que votre arbre de widgets commence à devenir trop grand, c'est une bonne idée de le diviser en widgets séparés.

Retournez dans **main.dart** pour le mettre à jour afin qu'il puisse utiliser le nouveau widget Home. En haut, ajoutez l'instruction d'import suivante :

```
import 'home.dart';
```

Ensuite, localisez **`TODO: Apply Home widget`** et remplacez-le ainsi que tout le **`home: Scaffold(...)`**, par ce qui suit :

```
home: Home(
  changeTheme: changeThemeMode,
  changeColor: changeColor,
  colorSelected: colorSelected,
),
```

Enfin, supprimez les imports suivants dans **main.dart** :

```
import 'components/theme_button.dart';
import 'components/color_button.dart';
```

Ils ne sont plus utilisés. Ceci fait, vous allez vous occuper de la barre de navigation inférieure de Scaffold.

Ajouter une Barre de Navigation Inférieure (BottomNavigationBar)

Ouvrez **home.dart**, localisez `// TODO: Track current tab` et remplacez-le par :

```
int tab = 0;
```

La propriété tab sera utilisée pour garder une trace de l'onglet actuel sur lequel l'utilisateur se trouve.

Votre prochaine étape consiste à définir une liste d'onglets entre lesquels l'utilisateur peut naviguer. Localisez `// TODO: Define tab bar destinations` et remplacez-le par le code suivant :

```
List<NavigationDestination> appBarDestinations = const [
  NavigationDestination(
    icon: Icon(Icons.credit_card),
    label: 'Category',
    selectedIcon: Icon(Icons.credit_card),
  ),
  NavigationDestination(
    icon: Icon(Icons.credit_card),
    label: 'Post',
    selectedIcon: Icon(Icons.credit_card),
  ),
  NavigationDestination(
    icon: Icon(Icons.credit_card),
    label: 'Restaurant',
    selectedIcon: Icon(Icons.credit_card),
  ),
];
```

Vous aurez un total de trois onglets où vous créerez trois widgets de carte distincts.

Enfin, localisez le commentaire `// TODO: Add bottom navigation bar` et remplacez-le par :

```
// 1
bottomNavigationBar: NavigationBar(
  // 2
  selectedIndex: tab,
  // 3
  onDestinationSelected: (index) {
    setState(() {
      tab = index;
    });
  },
  // 4
  destinations: appBarDestinations,
),
```

Voici comment le code fonctionne :

1. Assigne **`NavigationBar`** à **`bottomNavigationBar`**.
2. Définit l'onglet actif en utilisant **`selectedIndex`**.
3. Met à jour l'onglet actif lors de la sélection par l'utilisateur.
4. Définit la liste des onglets avec **`appBarDestinations`**.

Une fois cela fait, votre application devrait ressembler à ceci :

Figure 13 : Application avec barre de navigation inférieure

Maintenant que vous avez configuré la barre de navigation inférieure, vous devez implémenter la navigation entre les pages.

Naviguer Entre les Pages

Pour naviguer entre les pages, vous devez d'abord définir la liste des pages vers lesquelles l'utilisateur peut potentiellement naviguer. Toujours dans **home.dart**, localisez le commentaire **`// TODO: Define pages`** et remplacez-le par :

```
final pages = [
  // TODO: Replace with Category Card
  Container(color: Colors.red),
  // TODO: Replace with Post Card
  Container(color: Colors.green),
  // TODO: Replace with Restaurant Landscape Card
  Container(color: Colors.blue)
];
```

Cela contient une liste de conteneurs avec des couleurs différentes. Vous remplacerez chacun par une carte unique bientôt.

Ensuite, localisez le commentaire **`// TODO: Switch between pages`** et remplacez-le ainsi que tout le code **`body: Padding(...)`** par :

```
body: IndexedStack(
  index: tab,
  children: pages,
),
```

IndexedStack empile et affiche un widget de pages en fonction de l'index de l'onglet, préservant l'état de tous les widgets dans la pile.

Après le redémarrage, votre application aura un aspect différent pour chaque élément d'onglet, comme ceci :

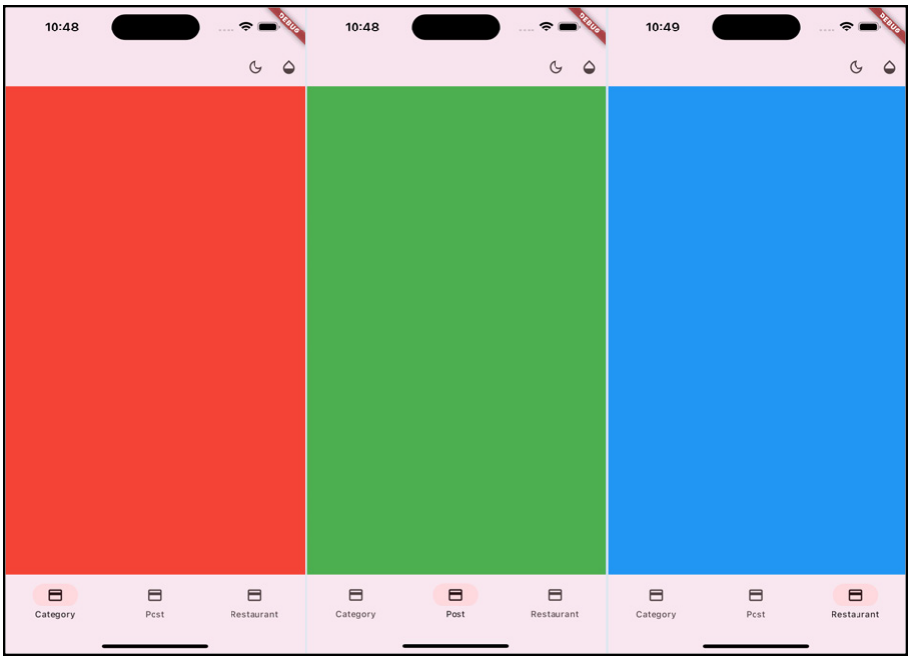

Figure 14 : Affichage des trois onglets avec des couleurs différentes

Maintenant que vous avez configuré votre navigation par onglets, il est temps de créer de belles cartes !

Créer des Cartes Personnalisées

Dans cette section, vous allez composer trois cartes en combinant un mélange de widgets d'affichage et de mise en page.

Remarque : Pour vous aider à construire ces cartes, le dossier **models** contient déjà des modèles et des données fictives pour chaque modèle à utiliser pour l'affichage dans vos widgets personnalisés.

Jetez un œil à **food_category.dart**, **post.dart** et **restaurant.dart** pour en savoir plus !

Les widgets **d'affichage** gèrent ce que l'utilisateur voit à l'écran. Les exemples de widgets d'affichage incluent :

- Text
- Image
- Button

Les widgets **de mise en page** aident à l'agencement des widgets. Les exemples de widgets de mise en page incluent :

- Container
- Padding
- Stack
- Column

- SizedBox
- Row

> **Remarque** : Flutter propose une pléthore de widgets de mise en page parmi lesquels choisir, mais cette leçon ne couvre que les plus courants. Pour plus d'exemples, consultez https://flutter.dev/docs/development/ui/widgets/layout.

Composer la Carte de Catégorie

La première carte que vous allez composer ressemble à ceci :

Figure 15 : Aperçu de la CategoryCard

`CategoryCard` est composée des widgets suivants :

- `Card` : Un conteneur de design material qui contient du contenu et des actions sur un seul sujet.
- `Column` : Arrange verticalement ses enfants widgets, ici, un Stack et un ListTile.
- `ListTile` : Un widget qui contient un titre et un sous-titre.

Dans le répertoire **lib/components**, créez un nouveau fichier appelé **category_card.dart** et ajoutez le code suivant :

```
import 'package:flutter/material.dart';
import '../models/food_category.dart';

class CategoryCard extends
StatelessWidget {
  // 1
  final FoodCategory category;
  const CategoryCard({ super.key,
    required this.category,
  });
  @override
  Widget build(BuildContext context) {
    // TODO: Get text theme
    // TODO: Replace with Card widget
    return Container(); // 2
  }
}
```

Ici, vous configurez la structure de base pour le widget **`CategoryCard`**.

1. Il prend simplement un objet **`FoodCategory`**, que vous utiliserez plus tard pour afficher les données dans votre interface utilisateur.
2. Renvoie un Container vide.

Ensuite, ouvrez **home.dart** et ajoutez l'import suivant :

```
import 'components/category_card.dart';
import 'models/food_category.dart';
```

Localisez **`// TODO: Replace with Category Card`** et remplacez le premier conteneur par :

```
// 1
Center(
  // 2
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 300),
    // 3
    child: CategoryCard(category: categories[0]),),),
```

Prenez un moment pour voir comment **`CategoryCard`** est disposée sur la page.

1. Le widget **`Center`** garantit que la carte est centrée à l'écran.
2. Applique une largeur maximale de **`300`** pixels à la carte.
3. Définit le widget **`CategoryCard`** comme enfant et passe la première catégorie fictive à afficher.

Vous avez maintenant configuré **`CategoryCard`**. Effectuez un redémarrage à chaud et l'application ressemblera actuellement à ceci :

Figure 16 : CategoryCard initiale vide

C'est un peu fade, n'est-ce pas ? Pour votre prochaine étape, vous allez l'égayer avec une image.

Construction du Widget

Passez à **category_card.dart**. Localisez **`// TODO: Get text theme`** et remplacez-le par :

```
final textTheme = Theme.of(context)
    .textTheme
    .apply(displayColor:
Theme.of(context).colorScheme.onSurface);
```

Ici, vous obtenez le thème de texte, que vous utiliserez plus tard pour l'appliquer aux widgets de texte.

Ensuite, localisez **`// TODO: Replace with Card widget`** et remplacez-le ainsi que le code en dessous par :

```
// 1
return Card(
  // 2
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: [
      // TODO: Add Stack Widget
      // TODO: Add ListTile widget
    ],
  ),
);
```

Vous remplacez le widget **`Container`** par un widget Card, et à l'intérieur de la carte, vous utilisez un widget **`Column`** pour organiser verticalement les widgets enfants.

Ajout d'Éléments Empilés à la Carte

Ajout du premier widget à la colonne. Localisez **`// TODO: Add Stack Widget`** et remplacez-le par :

```
Stack(
  children: [
    // 1
    ClipRRect(
      borderRadius: const
        BorderRadius.vertical( top:
        Radius.circular(8.0)),
      child: Image.asset(category.imageUrl),
    ),
    // 2
    Positioned(
      left: 16.0,
      top: 16.0,
      child: Text(
        'Yummy',
        style: textTheme.headlineLarge,
      ),
    ),
```

```
      // 3
      Positioned(
        bottom:
        16.0,
        right: 16.0,
        child:
        RotatedBox(
          quarterTurns: 1,
          child: Text(
            'Smoothies',
            style: textTheme.headlineLarge,
          ),
        ),
      ),
    ],
  ),
```

Rappelez-vous que le widget **`Stack`** vous permet de superposer des widgets les uns sur les autres.

Voici ce qui est superposé dans la pile :

1. Ajoutez un widget **`ClipRRect`**, qui rogne l'image avec des coins arrondis en haut.
2. Positionne le texte "**Yummy**" en haut à gauche.
3. Fait pivoter le texte "**Smoothies**" de 90 degrés et le place en bas à droite.

Après un redémarrage à chaud, la **`CategoryCard`** ressemble maintenant à ceci :

Figure 17 : CategoryCard avec image et texte superposé

Ajout d'un Pied de Page à la Carte

Ajout du deuxième widget à la colonne. Localisez `// TODO: Add ListTile widget` et remplacez-le par :

```
ListTile(
  // 1
  title: Text(
      category.name,
      style: textTheme.titleSmall,),
  // 2
  subtitle: Text(
      '${category.numberOfRestaurants}
      places', style:
      textTheme.bodySmall,),),
```

Prenez un moment pour parcourir le code :

1. Affiche le nom de la catégorie avec un style de titre plus petit.
2. Affiche le nombre de restaurants dans un style de texte de petit corps.

Après ces mises à jour, la `CategoryCard` finale ressemble à ceci :

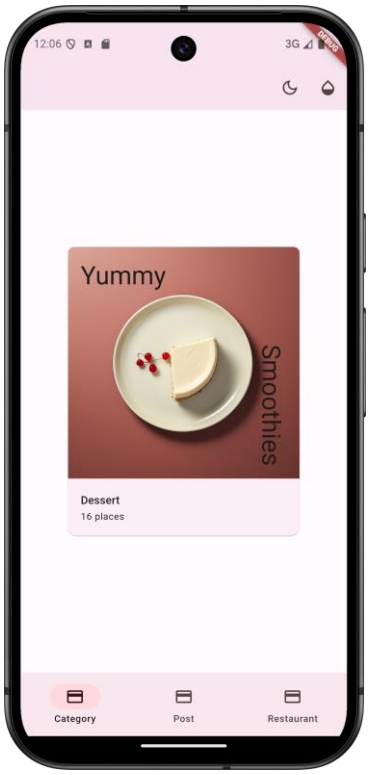

Figure 18 : CategoryCard finale

Super, vous avez terminé la première carte ! Il est temps de passer à la suivante !

> **Astuce** : Utilisez le thème de texte typographique de Material 3 pour des styles de texte cohérents dans toute votre application, en évitant les tailles de police et les couleurs codées en dur.

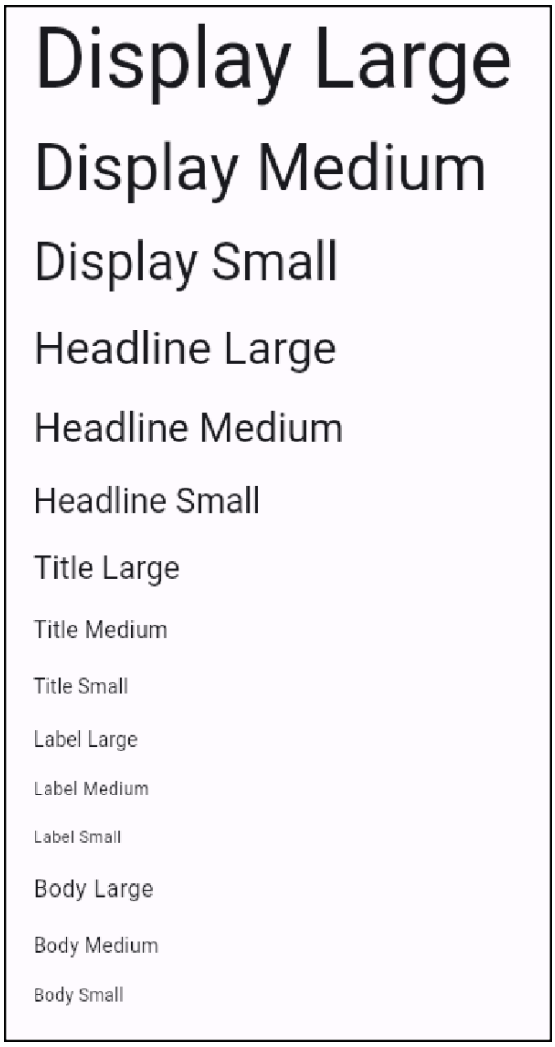

Figure 19 : Guide des styles de texte Material 3

Composer la Carte de Publication (Post Card)

Il est temps de commencer à composer la carte suivante, la carte de publication.

Voici à quoi elle ressemblera une fois terminée :

Figure 20 : Aperçu de la PostCard

`PostCard` est composée des widgets suivants :

- **`Card`** : Fournit une carte de design material qui peut contenir des informations ou du contenu liés.
- **`Padding`** : Ajoute une marge uniforme de 16,0 pixels autour du contenu à l'intérieur pour fournir un espacement.

Cette structure assure une mise en page propre et organisée où l'avatar de l'utilisateur est affiché à côté du contenu de sa publication, avec le commentaire et l'horodatage de la publication présentés en dessous.

Dans le répertoire **lib/components**, créez un nouveau fichier appelé **post_card.dart**. Ajoutez le code suivant :

```
import 'package:flutter/material.dart';
import '../models/post.dart';

class PostCard extends StatelessWidget {
  final Post post;

  const PostCard({ super.key,  required
    this.post,

  });
  @override

  Widget build(BuildContext context) {
    final textTheme = Theme.of(context)

        .textTheme

        .apply(

          displayColor:
          Theme.of(context).colorScheme.onSurface,

        );

    return Card( child: Padding(

        padding: const EdgeInsets.all(16.0),

        child: Row(

          crossAxisAlignment: CrossAxisAlignment.start,
          children: [

            // TODO: Add CircleAvatar

            // TODO: Add spacing

            // TODO: Add Expanded Widget

          ],

        ),

      ),

    );

  }

}
```

Ici, vous configurez la structure de base pour le widget **`PostCard`**. Il prend simplement un objet Post, que vous utiliserez plus tard pour afficher les données dans votre interface utilisateur. Ensuite, ouvrez **home.dart** et ajoutez l'import suivant :

```
import 'components/post_card.dart';
import 'models/post.dart';
```

Localisez **`// TODO: Replace with Post Card`** et remplacez le conteneur en dessous par :

```
Center(child: Padding(
  padding: const
  EdgeInsets.all(16.0), child:
  PostCard(post: posts[0]),
),),
```

Ensuite, effectuez un redémarrage à chaud.

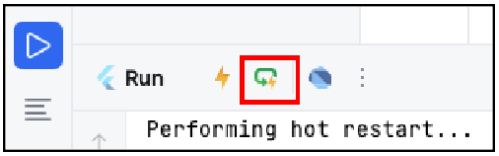

Figure 21 : PostCard initiale

Appuyez sur l'élément de la barre d'onglets **Post**. Votre application devrait ressembler à ceci :

Figure 22 : Onglet Post affichant la PostCard

Ajout des Widgets Enfants

Voici à quoi ressemblera la disposition de **`PostCard`** après avoir ajouté les widgets enfants de Row :

Figure 23 : Structure de disposition de la PostCard

Dans **post_card.dart**, localisez **`// TODO: Add CircleAvatar`** et remplacez-le par :

```
CircleAvatar(
  radius: 25,
  backgroundImage: AssetImage(post.profileImageUrl),
),
```

`CircleAvatar` est souvent utilisé pour afficher une image de profil ou l'avatar d'un utilisateur sous une forme circulaire.

Ensuite, localisez **`// TODO: Add spacing`** et remplacez-le par :

```
const SizedBox(
  width: 16.0,
),
```

Ajoutez un espacement de 16 pixels entre les deux widgets.

Enfin, localisez **`// TODO: Add Expanded Widget`** et remplacez-le par :

```
// 1
Expanded(
  // 2
  child: Column(
    mainAxisSize: MainAxisSize.min,
    crossAxisAlignment:
    CrossAxisAlignment.start, children: [
      // 3
      Text(
        post.comment,
        maxLines: 2,
        overflow: TextOverflow.ellipsis,
        style: textTheme.titleMedium),
```

```
        Text(
          '${post.timestamp} mins
          ago', style:
          textTheme.bodySmall,
        ),],),),
```

Voici ce que vous avez ajouté :

1. Le widget **`Expanded`** permet à l'enfant d'occuper tout l'espace disponible.
2. Le widget **`Column`** empile verticalement les enfants. **`MainAxisSize.min`** les aligne pour occuper un espace minimum. **`CrossAxisAlignment.start`** aligne horizontalement les widgets enfants sur le côté gauche.
3. Affiche deux widgets **`Text`**, le contenu de la publication suivi de l'horodatage de la publication.

Après un redémarrage à chaud, votre widget **`PostCard`** devrait ressembler à ceci :

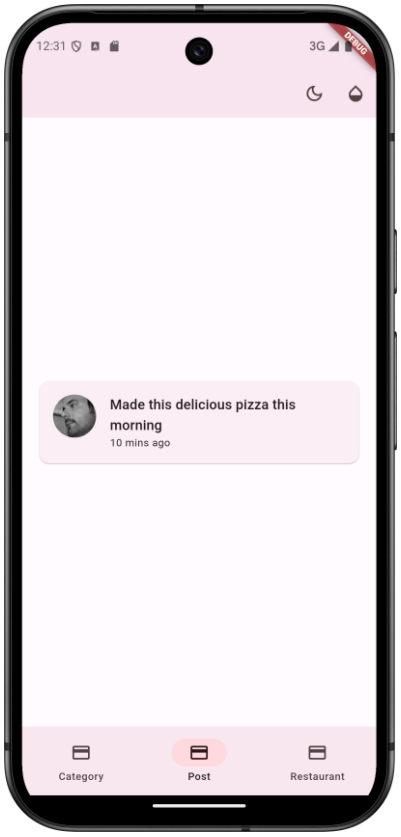

Figure 24 : PostCard terminée

Et c'est tout ce que vous avez à faire pour la carte de publication. Ensuite, vous passerez à la dernière.

Composer la Carte de Restaurant en Paysage

`RestaurantLandscapeCard` est la dernière carte que vous allez créer pour cette leçon. Cette carte permet à l'utilisateur d'explorer les tendances populaires des restaurants et de commander de la nourriture.

Figure 25 : Aperçu de la RestaurantLandscapeCard

Les widgets suivants composent **`RestaurantLandscapeCard`** :

- **`Card`** : Une carte de design material qui contient des informations connexes.
- **`Column`** : Arrange ses widgets enfants sur une ligne verticale.
- **`ClipRRect`** : Rogne son enfant avec un rectangle aux bords arrondis.
- **`AspectRatio`** : Contraint le rapport d'aspect de l'enfant.
- **`Image`** : Affiche l'image du restaurant, couvrant l'espace disponible.
- **`ListTile`** : Un widget qui contient un titre et un sous-titre.

Dans le répertoire **lib/components**, créez un nouveau fichier appelé **restaurant_landscape_card.dart**. Ajoutez le code suivant :

```
import 'package:flutter/material.dart';

import '../models/restaurant.dart';


class RestaurantLandscapeCard extends StatelessWidget
  { final Restaurant restaurant;

  const RestaurantLandscapeCard({
    super.key,

    required this.restaurant,

  });
```

```
@override
Widget build(BuildContext
  context) { final textTheme =
  Theme.of(context)
      .textTheme
      .apply(
        displayColor: Theme.of(context)
        .colorScheme
        .onSurface);
  return Card(
    child: Column(
      mainAxisSize:
      MainAxisSize.min, children:
      [
        // TODO: Add Image
        // TODO: Add ListTile
      ],),);}}
```

Ici, vous configurez la structure de base pour le widget **`RestaurantLandscapeCard`**. Il prend simplement une instance de Restaurant, que vous utiliserez plus tard pour afficher les données dans votre interface utilisateur.

Ensuite, ouvrez **home.dart** et ajoutez l'import suivant :

```
import 'components/restaurant_landscape_card.dart';
import 'models/restaurant.dart';
```

Localisez **`// TODO: Replace with Restaurant Landscape Card`** et remplacez le conteneur en dessous par :

```
// 1
Center(
  //2
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 400),
    // 3
    child: RestaurantLandscapeCard(
      restaurant:
      restaurants[0],),),),),
```

Prenez un moment pour voir comment **`RestaurantLandscapeCard`** est disposée sur la page :

1. Le widget **`Center`** garantit que la carte est centrée à l'écran.
2. Applique une largeur maximale de **`400`** pixels à la carte.
3. Définit le widget **`RestaurantLandscapeCard`** comme enfant et passe le premier restaurant fictif à afficher.

Maintenant que vous avez configuré **`RestaurantLandscapeCard`**, effectuez un redémarrage à chaud. Appuyez sur l'élément de la barre d'onglets **Restaurant**.

Votre application devrait ressembler à ceci :

Figure 26 : Onglet Restaurant affichant la carte de base

Composition des Widgets Enfants du Restaurant

Ouvrez **restaurant_landscape_card.dart**, localisez **`// TODO: Add Image`** et remplacez-le par :

```
ClipRRect(
  // 1
  borderRadius:
      const BorderRadius.vertical(top: Radius.circular(8.0),),
  // 2
  child: AspectRatio( aspectRatio: 2,
      child: Image.asset(restaurant.imageUrl, fit:
BoxFit.cover,),),),
```

Voici comment l'image est formée :

1. **`borderRadius`** arrondit les coins supérieurs avec un rayon de 8,0 unités.
2. **`AspectRatio`** affiche une image avec un rapport largeur/hauteur de 2:1. L'image s'adapte pour remplir son conteneur.

Ensuite, localisez **`// TODO: Add ListTile`** et remplacez-le par :

```
ListTile(
  // 1
  title: Text(restaurant.name, style:
  textTheme.titleSmall,),
  // 2
  subtitle: Text(restaurant.attributes,
      maxLines: 1, style:
      textTheme.bodySmall,),
  // 3
  onTap: () {
    // ignore: avoid_print
    print('Tap on ${restaurant.name}');
  },),
```

Le code représente un **`ListTile`** dans Flutter :

1. **`title`** affiche le nom du restaurant avec un style spécifique.
2. **`subtitle`** affiche les attributs du restaurant, tronqués s'ils dépassent une ligne.
3. **`onTap`** imprime le nom du restaurant sur la console lorsqu'il est tapé.

Enregistrez vos modifications et effectuez un redémarrage à chaud. Maintenant, votre carte ressemble à ceci :

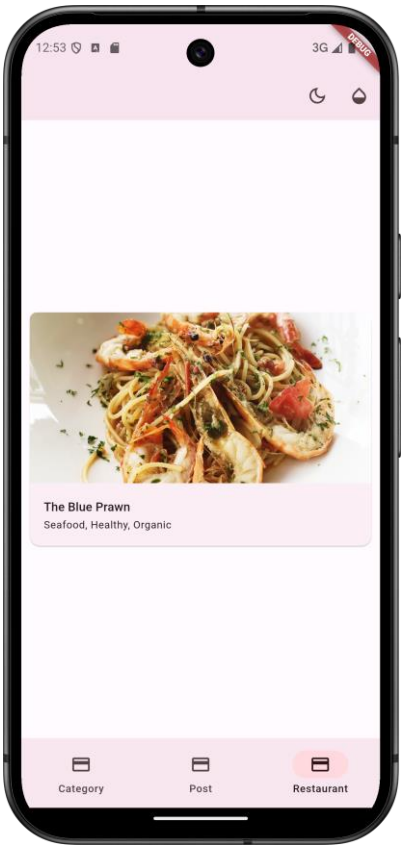

Figure 27 : RestaurantLandscapeCard terminée

Maintenant que Yummy est un StatefulWidget, vous pouvez vérifier que les déclarations **`const`** sont rajoutées. Ouvrez **main.dart** et changez :

```
runApp(Yummy());
```

en

```
runApp(const Yummy());
```

Puis changez :

```
Yummy({super.key});
```

en

```
const Yummy({super.key});
```

Lorsque vous avez terminé votre application, il reste une dernière étape : supprimer le label **Debug**.

Toujours dans **main.dart**, trouvez **`// debugShowCheckedModeBanner: false,`** et supprimez le **`//`** au début de la ligne. Effectuez un redémarrage à chaud de votre application, et la bannière a disparu.

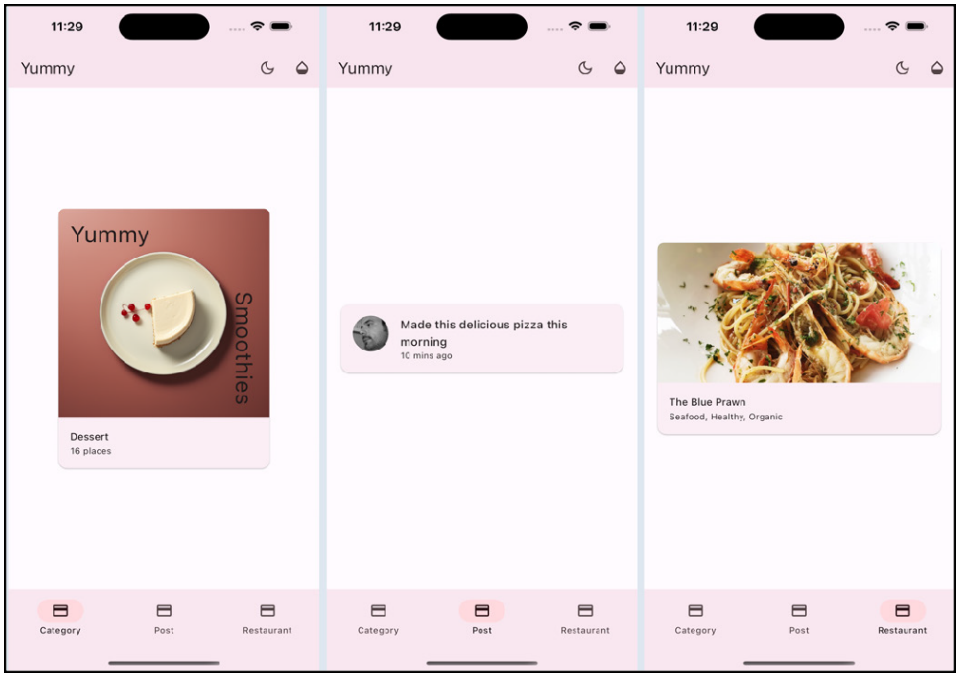

Figure 28 : Application finale sans la bannière de débogage

Vous l'avez fait ! Vous avez terminé cette leçon. En chemin, vous avez appliqué trois catégories différentes de widgets. Vous avez appris à utiliser des widgets structurels pour organiser différents écrans, et vous avez créé trois cartes personnalisées et appliqué différentes mises en page de widgets à chacune.

Bien joué !

Points Clés

- Les trois principales catégories de widgets sont : la structure et la navigation, l'affichage d'informations et le positionnement des widgets.
- Il existe deux principaux systèmes de design visuel disponibles dans Flutter, **Material** et **Cupertino**. Ils vous aident à créer des applications qui ont l'air natives sur Android et iOS, respectivement.
- En utilisant le thème **Material**, vous pouvez construire des éléments d'interface utilisateur assez variés pour donner à votre application un aspect et une convivialité personnalisés.
- C'est généralement une bonne idée d'établir un objet de thème commun pour votre application, vous donnant une source unique de vérité pour le style de votre application.
- Le widget **Scaffold** implémente tous vos besoins de base en matière de structure de mise en page visuelle.
- Le widget **Container** peut être utilisé pour regrouper d'autres widgets.
- Le widget **Stack** superpose les widgets enfants les uns sur les autres.